

Shipment Integrity Platform

Technical Strategy, Architecture and Delivery Plan

Senior .NET / C# Technical Assessment | .NET 10 | Microsoft SQL Server | Azure-oriented production design

1. Problem, Scope, Requirements and Ownership Boundaries

Problem. Courier webhook events may arrive late, out of order, duplicated or with conflicting data. Downstream systems therefore disagree about shipment state. The goal is to create one reliable current view while preserving a queryable history that explains how that view was derived.

In scope

- Receive and validate DHL shipment events over webhook.
- Normalize partner data into a stable internal event model.
- Detect duplicates, out-of-order events and conflicting transitions.
- Persist event history and derive a trusted current ShipmentState.
- Expose current state/history to internal consumers and support operations.
- Provide observability, retry/failure handling, controlled rollout and rollback.

Deliberately postponed

- Customer-facing UI and admin conflict UI.
- Additional courier partners beyond DHL.
- Full event-sourcing platform, Kafka, advanced analytics/ML and multi-region active-active unless justified by client requirements.

Assumptions to Validate and Open Questions for the Client

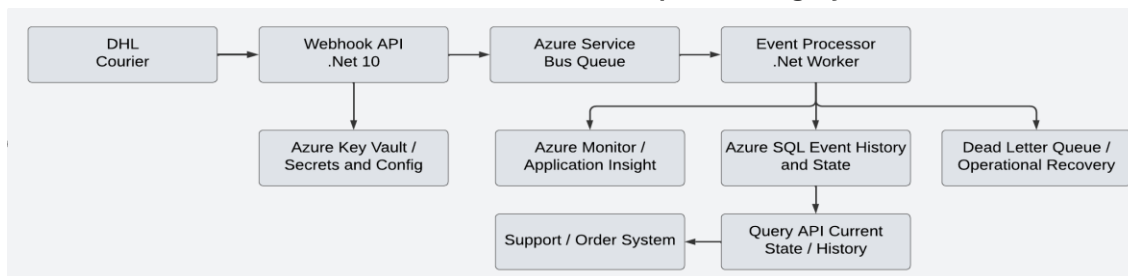
Assumption / question	Why it matters
partner and eventId can identify one logical event; confirm uniqueness guarantees.	Defines idempotency key and database constraint.
occurredAt is the authoritative business timestamp; confirm correction/sequence semantics.	Determines ordering and stale-event handling.
Authoritative DHL transition rules are not supplied.	The transition matrix must be confirmed before production.
Expected average/peak events, retry behaviour and SLA are unknown.	Required to size queue, processors and SQL Server.
Retention, Recovery Time Object and Recovery Point Object (RPO/RTO), webhook authentication and ownership must be confirmed.	Affects security, operations, cost and recovery.

Ownership Boundaries

Area	Owner
Courier event contract and credentials	Client integration/platform team
Shipment transition business rules	Client product/business owner
API, processor and integrity logic	Application engineering team
SQL Server / Service Bus platform	Cloud/platform team
Monitoring, DLQ and production incidents	Application operations/on-call
Courier-side delivery failures	DHL, escalated via client integration owner

2. Proposed Production Architecture

Production Architecture – Shipment Integrity



Immutable event history and determine current-state projection

Decision. Separate durable ingestion from state processing. The .NET 10 webhook boundary validates/normalizes events and publishes them to Azure Service Bus. A .NET processor applies deterministic integrity rules and stores both immutable event history and the current projection in Microsoft SQL Server/Azure SQL.

Trade-Offs

Decisions	Benefits	Trade-offs / Costs
Service Bus vs synchronous processing	Absorbs bursts; protects webhook availability	Eventually consistency and operational complexity
MSSQL vs Cosmos DB	Transactions, uniqueness, concurrency and familiar querying	Less elastic/global than Cosmos
Service Bus vs Kafka	Simpler fit for reliable command/event processing	Less appropriate if large-scale event streaming later becomes necessary
Current-state projection + history vs full event sourcing	Simple, auditable and pragmatic	Replay/versioning capabilities are more limited
No UI initially	Focus effort on highest-risk integrity problem	Operations may later require conflict-management tooling

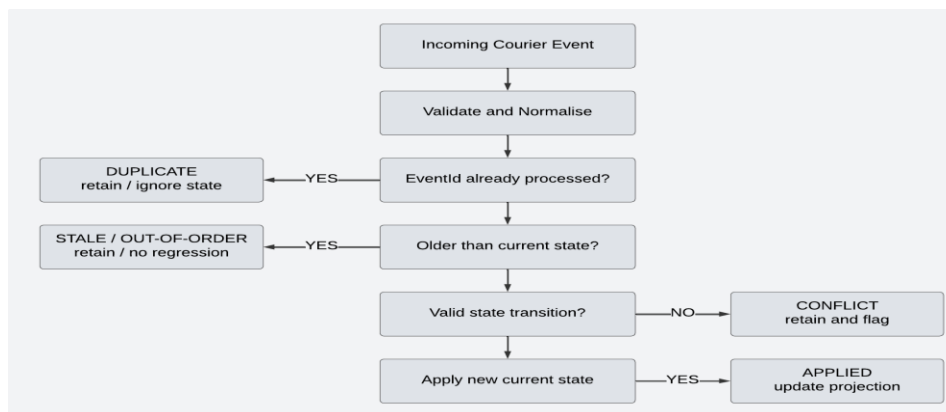
Key Architecture Trade-offs

Why not a UI, Kafka or Cosmos?

A UI is not required to prove the integrity problem. Kafka is not justified without evidence of streaming-scale requirements. SQL Server is preferred for the first release because uniqueness, transactions, concurrency control and relational history/state queries are central to the problem.

3. Data Integrity and Processing Strategy

Event Integrity Decision Flow



Event Integrity Decisions

Case	Rule	State effect
Duplicate	Same partner and eventId. Database uniqueness is the final guard.	No repeated business effect.
Out of order / stale	Incoming occurredAt is older than current projection timestamp.	Retain event; do not regress current state.
Conflict	Newer events violate approved transition rules, or same timestamp has conflicting statuses.	Retain and flag; trusted state unchanged.
Applied	New events are chronologically valid and transition is allowed.	Persist event and update current state atomically.

SQL Server integrity boundary

Core tables: Shipment, ShipmentEvent and ShipmentState.

ShipmentEvent retains the courier evidence and processing outcome; ShipmentState is the fast current projection. Production persistence should enforce UNIQUE(Partner, EventId), foreign keys, a transaction around event insert and state update, and ROWVERSION/optimistic concurrency for competing processors.

Failure behaviour

- Do not acknowledge a webhook as durably accepted until the queue/message boundary succeeds.
- Transient processor/database failures retry safely because processing is idempotent.
- After retry threshold, isolate the message in Dead-Letter Queue (DLQ) and alert operations.
- Do not delete historical events during recovery; rebuild/correct the projection from retained evidence when needed.

4. Delivery, Scrum, Refinement and Release Control

Agile Delivery and Release Control Flow



Every release has acceptance evidence, observability checks, rollback readiness and an explicit go/no-go decision

Scrum flow

Plan the smallest vertical slice → develop/test continuously → Daily Standups around sprint goal/blockers → review working integrity scenarios → retrospective → release readiness. The first sprint should prove the end-to-end path rather than complete isolated technical layers.

Refinement

- Start from business failure scenarios: duplicate, late/out-of-order, conflict and normal progression.
- Clarify acceptance criteria, unknowns and dependencies before development.
- Split work vertically so each item is testable end to end.
- Identify security, retry, observability and concurrency requirements during refinement, not after coding.
- A refined item is ready when the team can explain success, test evidence and remaining dependencies.

Release control

Stage	Gate
Development / thin slice	Domain/unit tests pass; local MSSQL integration works.
Integration	Webhook, SQL and messaging integration and security checks pass.
Staging	Production-like failure/performance tests, dashboards and runbook validated.
Limited production	Controlled traffic; integrity metrics and DLQ stable.
Full release	Business/technical go-no-go, on-call ownership and rollback readiness confirmed.

Rollback plan

Stop harmful processing first, preserve event history, redeploy the previous well-good application version, validate current state, then replay/rebuild only after the faulty rule is understood. Database changes should use backward-compatible expand/contract migrations, so application rollback remains possible.

5. Delivery and Test Plan, Risks and Success Signals

Delivery Plan

Phase	What ships
1. Discovery / refinement	Confirm courier semantics, transition rules, security, load assumptions and acceptance examples.
2. Thin executable slice	.NET 10, C# domain processor, MSSQL and tests proving applied/duplicate/stale/conflict behaviour.
3. Production integration	Service Bus, SQL persistence, query endpoints, authentication, concurrency and integration tests.
4. Hardening	Observability, failure/performance tests, runbook, release/rollback rehearsal.
5. Controlled production	Staging → limited production → full traffic using release gates.

Test plan

Testing will focus on proving shipment-state integrity and safe failure behaviour rather than broad API coverage. Automated tests will run in CI before deployment, with integration and failure-path validation completed before production release.

Test Plan Summary

Test level	Key validation	Expected results
Unit	Normal transition, duplicate, stale/out-of-order, same-time conflict, invalid transition	Deterministic APPLIED, DUPLICATE, STALE, or CONFLICT result
API	Valid/invalid DHL webhook payloads, status codes, serialization	Contract validated; invalid requests rejected safely
Integration	ADO.NET and MSSQL persistence, unique (Partner, EventId), transactions	History and current state remain consistent
Concurrency	Same shipment/event processed concurrently	No duplicate event or lost state update
Failure/Recovery	SQL unavailable, processing failure, retry/DLQ path	No silent event loss; recovery is observable
Performance	Normal load, peak burst and backlog recovery	Processing latency and queue age remain within agreed targets
Release/Smoke	Health endpoint, webhook, state query and history query after deployment	Release accepted only when critical flow succeeds

Debt management:

Technical debt will be recorded in the delivery backlog with an owner, rationale and review trigger. Items will be prioritised based on production evidence, operational risk and business value rather than automatically being treated as immediate defects.

Deferred item	Reason deferred	Follow-up
Single courier implementation	DHL is currently the only partner; premature abstraction adds complexity.	Introduce partner adapters when a second courier is onboarded.
Transition rules in application code	Keeps the initial rules explicit, testable and simple.	Externalise/configure rules if business changes become frequent.
No operational conflict UI	Not required to prove the integrity solution.	Add support tooling if conflict volumes justify it.
Limited replay/reconciliation tooling	Initial focus is reliable ingestion and state derivation.	Add controlled replay/rebuild capability based on operational need.
Local Docker Compose deployment	Provides reproducible development/testing, not production orchestration.	Production deployment follows the client's Azure/container platform standards.

CI/CD Pipeline

Delivery will use an Azure DevOps CI/CD pipeline to provide repeatable build, test and deployment controls. Every pull request triggers continuous integration; deployment to higher environments occurs only after automated quality gates succeed.

PR → Restore – Build .Net 10 → Unit Tests → Integration Tests → Security / Quality Checks → Publish Versioned Artifact → Deploy Staging → Smoke Test → Approval Gate → Production → Health Check and Monitor.

Minimum credible first production slice

One courier; authenticated webhook; durable queue; deterministic integrity processing; MSSQL event history and current-state projection; minimal query API; automated tests; observability; retry/DLQ; rollback/recovery procedure.

Top risks and mitigations

Risk	Mitigation
Wrong/ambiguous courier transition rules	Client sign-off, table-driven rules and comprehensive transition tests.
Duplicate or concurrent processing	UNIQUE(Partner, EventId), transaction boundary and row version concurrency.
Traffic burst or partner retry storm	Service Bus buffering, worker scaling and queue-age alerts.
Poison/invalid event	Bounded retries, DLQ, structured reason and operational owner.
Bad release corrupts projection	Preserve event history, stop processing, rollback app, rebuild projection safely.

Success signals

- No duplicate-driven state changes and no regressions caused by late events.
- Every accepted event is classified as APPLIED, DUPLICATE, STALE or CONFLICT; no silent loss.
- P95 event-to-current-state latency stays within the agreed SLA.
- Conflict and DLQ rates are visible, owned and within agreed thresholds.
- Current state can be explained from retained event history and recovered after rollback.

Thin slice to implementation

Build .NET 10 and ADO (SqlConnection) and Microsoft SQL Server with POST /api/webhooks/dhl, GET /api/shipments/{id}, GET /api/shipments/{id}/events, domain rules, xUnit tests and integration tests.

The executable slice proves the highest risk integrity behaviour;

The production architecture remains the target deployment model.